

## Schema-less, document-oriented Open-Source-Database

Name comes from English *humongous*, „gigantic“

## Widespread NoSQL DBMS

No DDL (Data Definition Language)

Manages collections of JSON-based documents

Stores BSON (Binary JSON) Objects

Data can be nested in complex hierarchies

Developed by 10gen (today MongoDB, Inc.), established 2007

Written in C++

API-support for many programming languages

JavaScript, Python, Ruby, Perl, Java, JavaScala, C#, C++, Haskell,  
Erlang



**Dynamic** schema

No DDL

**Document-based** database

**Simple query language** via an API

**Master-slave replication** with automated failover (replica sets)

**Built-in horizontal** scaling via automated range-based partitioning of data (**sharding**)

**No joins, only limited transactions**

atomic operations on a single document

**Atomic writes and fully-consistent reads**

If system configured that way

**Map-Reduce** Support

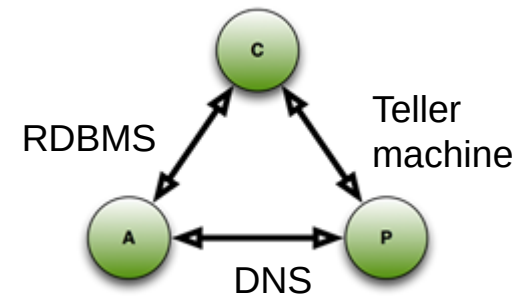
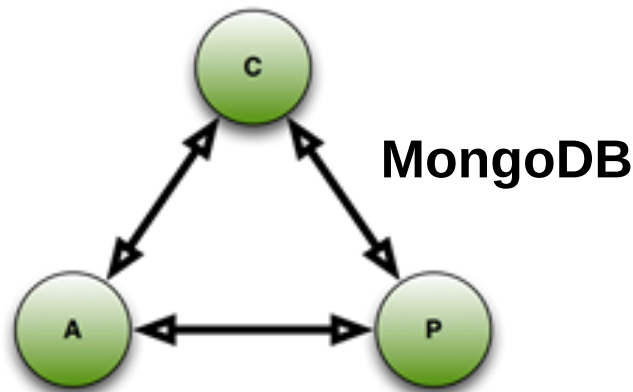


## Focus on Consistency (C) and Partition tolerance (P)

Consistency (C): all replicates contain the same version of data, user sees everywhere (independent of site) the same data

Partition tolerance (P): many access points to system, system stays operational with system-/network-split (communication error, split of network infrastructure)

Availability (A) NOT guaranteed: Not all clients can read or write always (with site error, system partition, ...)



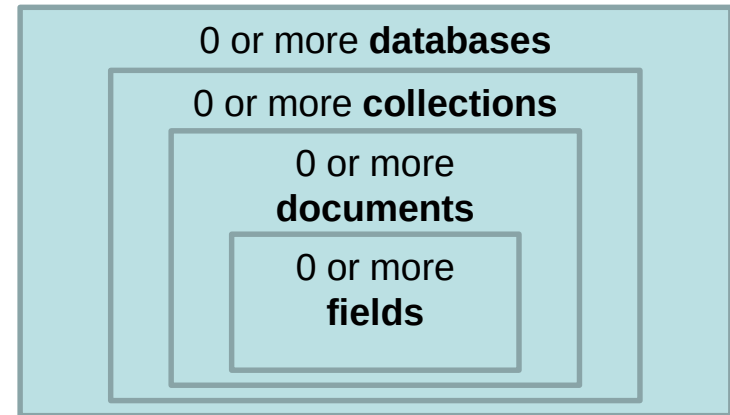
A MongoDB instance may have zero or more **databases**

A database may have zero or more **collections**

A collection may have zero or more **documents**

A document may have one or more **fields**

MongoDB **Indexes** function much like their RDBMS counterparts.



RDBMS-MongoDB concepts compared

| RDBMS             | MongoDB                                              |
|-------------------|------------------------------------------------------|
| Database          | Database                                             |
| Table, View       | Collection                                           |
| Row, Tuple        | Document                                             |
| Column, Attribute | Field                                                |
| Index             | Index                                                |
| Join              | Embedded document or reference                       |
| Primary key       | Default key <code>_id</code> is delivered by MongoDB |
| Foreign key       | Reference                                            |



## Start MongoDB server (Windows example, if not service)

```
C:\Program Files\MongoDB\Server\4.0\bin> mongod
```

```
...
```

```
2018-10-21T10:38:39.904+0200 I CONTROL [initandlisten] MongoDB starting : pid=7064  
port=27017 dbpath=C:\data\db\ 64-bit host=quickstep
```

```
...
```

## Start MongoDB shell

```
C:\Users\schiki> mongo
```

```
MongoDB shell version v4.2.1
```

```
connecting to: mongodb://127.0.0.1:27017/?compressors=disabled&gssapiServiceName=mongodb
```

```
Implicit session: session { "id" : UUID("d0dbb657-972f-4086-bb5b-8ab655f107c8") }
```

```
MongoDB server version: 4.2.1
```

```
...
```

```
>
```

## Create database mydb

```
> use mydb
```

```
switched to db mydb
```

```
>
```

## Show databases

```
> show dbs
```

```
admin    0.000GB
```

```
config   0.000GB
```

```
local    0.000GB
```

```
>
```

Database mydb is not shown, due to no document inserted yet



## Show selected database

```
> db
mydb
>
```

## Insert collection

```
> db.createCollection("students")
{ "ok" : 1 }
>
```

## List all collections

```
> show collections
students
>
```

## Collections are created automatically, if a document is inserted

```
> db.movie.insert({"name":"Star Trek Discovery"})
WriteResult({ "nInserted" : 1 })
> show collections
movie
students
>
```



## Create

```
db.collection.insert( <document> )
```

```
db.collection.save( <document> )
```

## Read

```
db.collection.find( <query>, <projection> )
```

```
db.collection.findOne( <query>, <projection> )
```

## Update

```
db.collection.update( <query>, <update>, <options> )
```

## Delete

```
db.collection.remove( <query>, <justOne> )
```

*collection* specifies the collection where the document is stored



```
> db.students.insert({
  _id:"4711",
  name:"Max Mustermann",
  regnr:"1600123",
  curriculum: [
    {
      name:"Business Informatics",
      type:"Bachelor",
      currnr:526
    },
    {
      name:"Studium Irregulare",
    }
  ]
})
WriteResult({ "nInserted" : 1 })
>
```

Documents are written in JSON

Primary key can be specified by field `_id`, otherwise it is generated automatically by MongoDB



## Simple queries by find() method

```
> db.students.find()
{ "_id" : "4711", "name" : "Max Mustermann", "regnr" : "1600123", "curriculum"
  : [ { "name" : "Wirtschaftsinformatik", "type" : "Bachelor", "currnr" : 526
    }, { "name" : "Studium Irregulare" } ] }
>
```

## Formatted query by pretty() method

```
> db.students.find().pretty()
{
    "_id" : "4711",
    "name" : "Max Mustermann",
    "regnr" : "1600123",
    "curriculum" : [
        {
            "name" : "Business Informatics",
            "type" : "Bachelor",
            "currnr" : 526
        },
        {
            "name" : "Studium Irregulare"
        }
    ]
}
>
```



## Insert of another document

```
> db.students.insert({name:"Ernst Müller",regnr:"1600124",curriculum:
  [{name:"Business Informatics",type:"Master",currnr:926}]})
WriteResult({ "nInserted" : 1 })
```

## Query all documents

```
> db.students.find()
{ "_id" : "4711", "name" : "Max Mustermann", "regnr" : "1600123", "curriculum" : [ {
  "name" : "Business Informatics", "type" : "Bachelor", "currnr" : 526 }, { "name"
  : "Studium Irregulare" } ] }
{ "_id" : ObjectId("5dc012455305f024e46ba928"), "name" : "Ernst Müller", "regnr" :
  "1600124", "curriculum" : [ { "name" : "Business Informatics", "type" : "Master",
  "currnr" : 926 } ] }
```

## Query with given field value

```
> db.students.find({"regnr":"1600123"})
{ "_id" : "4711", "name" : "Max Mustermann", "regnr" : "1600123", "curriculum" : [ {
  "name" : "Business Informatics", "type" : "Bachelor", "currnr" : 526 }, { "name"
  : "Studium Irregulare" } ] }
```

## AND, OR expressions possible

```
> db.students.find({$or:[{"regnr":"1600123"}, {"regnr":"1600124"}]})
...
```

## Support of arbitrary conditional operators (<, >, ...)



# MongoDB Java programming example: Connection and Access



```
import com.mongodb.MongoClient;
import com.mongodb.client.MongoDatabase;
import com.mongodb.client.MongoCollection;
import com.mongodb.client.MongoCursor;
import static com.mongodb.client.model.Filters.*;
import org.bson.Document;

public class ReadDB {
    public static void main( String args[] ) {
        // Creating a Mongo client
        MongoClient mongo = new MongoClient( "localhost" , 27017 );
        // Accessing the database
        MongoDatabase database = mongo.getDatabase("mydb");
        // Retrieving a collection
        MongoCollection<Document> collection = database.getCollection("students");
        // Count documents
        System.out.print(collection.count()); System.out.println(" Docs in collection");
        // Getting the iterator
        MongoCursor<Document> it = collection.find().iterator();
        try {
            while (it.hasNext()) { System.out.println(it.next()); }
        } finally { it.close(); }
        // Find specific document
        Document myDoc = collection.find(eq("regnr", "1600234")).first();
        System.out.println(myDoc.toJson());
    }
}
```



# MongoDB Java programming example:

## Insert and Delete



```
public class ManageDB {
    public static void main( String args[] ) {
        // Create a Mongo client, Access the database, Retrieve a collection
        ...
        // Enter new document
        Document doc = new Document("id", "4712")
            .append("name", "Sylvia Musterfrau")
            .append("regnr", "1600234")
            .append("curriculum", new Document("name", "Informatics")
                .append("type", "Bachelor")
                .append("currnr", "521" ));
        collection.insertOne(doc);
        // Enter new document list with JSON
        String json = "{ 'id': '4713', 'name': 'Lara Musterkind', 'regnr': '1600345',
            'curriculum' : [
                { 'name' : 'Informatics',
                  'type' : 'Master',
                  'currnr' : '921' },
                { 'name' : 'Studium Irregulare' }
            ] }";
        doc = Document.parse(json);
        collection.insertOne(doc);
        // Delete specific document
        collection.deleteMany( or( eq("regnr", "1600234"), eq("regnr", "1600345") ) );
    }
}
```



```
<?php
// connect
    $m = new MongoClient();
// select a database
    $db = $m->mydb;
// select a collection
    $collection = $db->students;
// add a record
    $collection->insert(array("name"=>"Tick", "regnr"=>"1600456"));
// add another record differently
    $document = array("id"=>"4714", "curriculum"=>"Informatics");
    $collection->insert($document);
// find all students of 'Informatics' in the collection
    $cursor = $collection->find(["curriculum"=>"Informatics"]);
// iterate through the results
    foreach ($cursor as $document) {
        echo $document["name"] . "\n";
    }
?>
```



# Migration: Examples of “Proven Benefits”



| Organization                  | Migrated From  | Application                            |
|-------------------------------|----------------|----------------------------------------|
| edmunds.com                   | Oracle         | Billing, online advertising, user data |
| Cisco                         | Multiple RDBMS | Analytics, social networking           |
| Craigslist                    | MySQL          | Content management                     |
| Salesforce<br>Marketing Cloud | RDBMS          | Social marketing, analytics            |
| Foursquare                    | PostgreSQL     | Social, mobile networking platforms    |
| MTV Networks                  | Multiple RDBMS | Centralized content management         |
| Orange Digital                | MySQL          | Content Management                     |

Siehe <https://www.mongodb.com/who-uses-mongodb>

Mat Keep, RDBMS to MongoDB Migration - Considerations and Best Practices

<https://de.slideshare.net/matkeep/migrating-from-relational-databases-to-mongodb>



## Relational

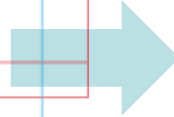
Person:

| Pers_ID | Surname   | First_Name | City     |
|---------|-----------|------------|----------|
| 0       | Miller    | Paul       | London   |
| 1       | Ortega    | Alvaro     | Valencia |
| 2       | Huber     | Urs        | Zurich   |
| 3       | Blanc     | Gaston     | Paris    |
| 4       | Bertolini | Fabrizio   | Rom      |

Car:

| Car_ID | Model       | Year | Value  | Pers_ID |
|--------|-------------|------|--------|---------|
| 101    | Bentley     | 1973 | 100000 | 0       |
| 102    | Rolls Royce | 1965 | 330000 | 0       |
| 103    | Peugeot     | 1993 | 500    | 3       |
| 104    | Ferrari     | 2005 | 150000 | 4       |
| 105    | Renault     | 1998 | 2000   | 3       |
| 106    | Renault     | 2001 | 7000   | 3       |
| 107    | Smart       | 1999 | 2000   | 2       |

no relation



## MongoDB Document

```
{  
  first_name: 'Paul',  
  surname: 'Miller',  
  city: 'London',  
  location: [45.123,47.232],  
  cars: [  
    { model: 'Bentley',  
      year: 1973,  
      value: 100000, ... },  
    { model: 'Rolls Royce',  
      year: 1965,  
      value: 330000, ... }  
  ]  
}
```

Join realized by embedded document or  
reference

Data aggregated in a single structure (pre-JOINED)



## Modeling of relationships

### **Nesting, Embedding**

With **1:1** or **1:n** relationships

Document limit of 16MB with growing documents has to be respected

### **Referencing**

With complex **n:m** relationships and hierarchical relationships

`_id` field is referenced in the related document

Application has to execute 2nd query to get data

## Advantages of document model

Rich data model, natural data representation

Embedding of related data in sub-documents or arrays

Data aggregated within one structure (pre-JOINED)

Programming is easier

Data duplication (redundancy) versus performance gain

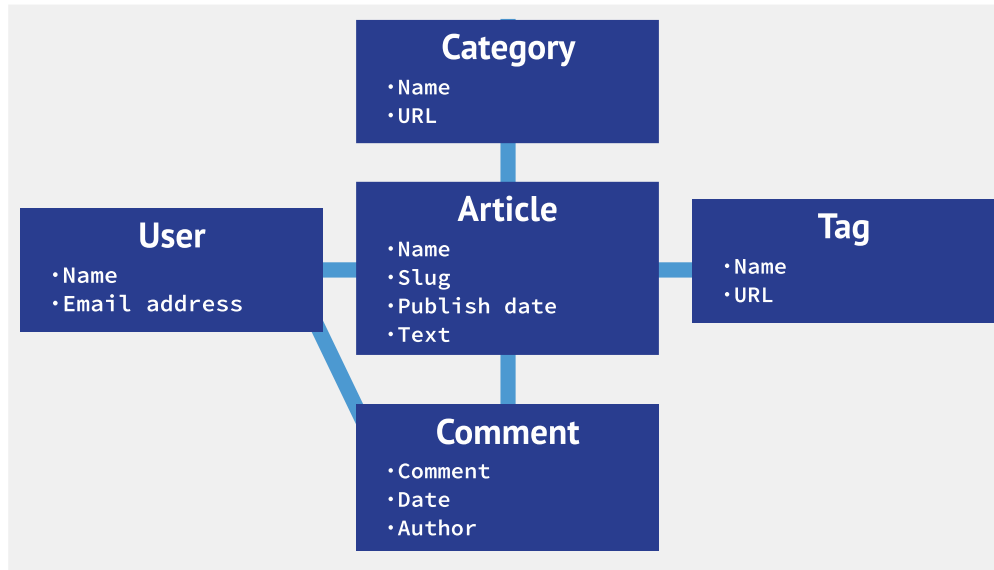
### **Dynamic Schema**

Data models easy to develop and extend

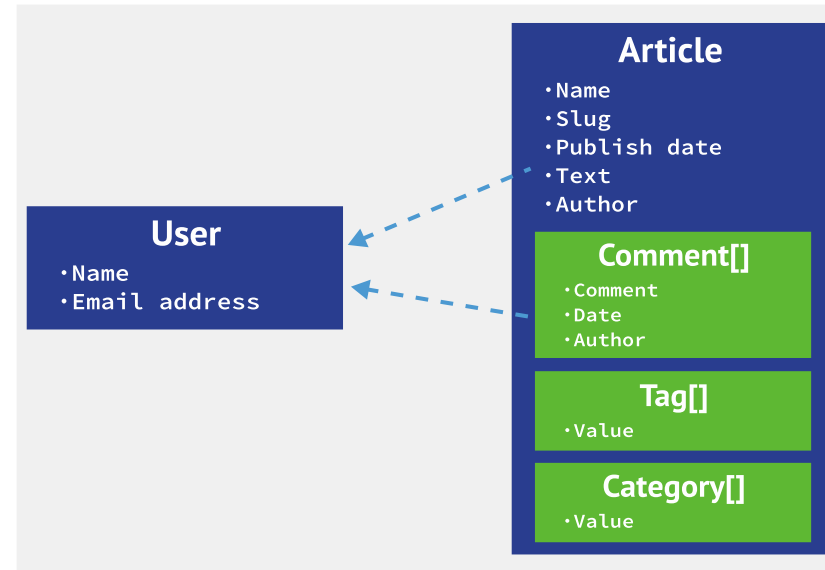
Quick adaptation of changes: Agile Methods



## RDBMS: JOIN 5 tables



## MongoDB: Nested documents



## RDBMS

Five tables, one Blog entry is built by a Join of 5 tables

## MongoDB

Single document with a link to a user document, which contains the Blog entry and the comment of user

Thus: Pre-JOINing to merge 5 RDBMS tables to 2 BSON documents

Advantage:

Complete document can be retrieved by one single access to the database (no Join necessary)

Stored as a single document, therefore only one read operation to memory or disk necessary

Distribution of database among several sites ("sharding") is simpler and allows massive horizontal scalability



## Website

<https://www.mongodb.com>

## Tutorials/White Papers

<https://docs.mongodb.org/manual/tutorial/>

<https://www.mongodb.com/white-papers/>

<http://www.tutorialspoint.com/mongodb/>

## Java MongoDB

<http://mongodb.github.io/mongo-java-driver/3.0/driver/getting-started/quick-tour/>

[https://www.tutorialspoint.com/mongodb/mongodb\\_java.htm](https://www.tutorialspoint.com/mongodb/mongodb_java.htm)

<http://mongodb.github.io/mongo-java-driver/3.5/javadoc/>

## PHP MongoDB

<http://php.net/manual/de/mongo.tutorial.php>

<https://www.sitepoint.com/building-simple-blog-app-mongodb-php/>

## RDBMS to MongoDB Migration

<https://www.mongodb.com/collateral/rdbms-mongodb-migration-guide>

